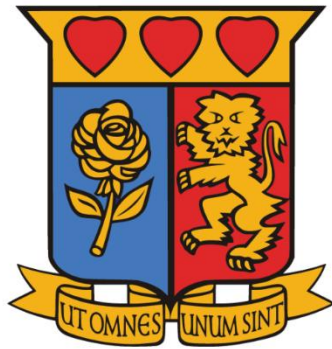




Strathmore
UNIVERSITY

SCHOOL OF COMPUTING AND ENGINEERING SCIENCES

BACHELOR OF SCIENCE IN ELECTRICAL AND ELECTRONICS ENGINEERING

BEE 5102: Embedded Systems & IoT

Group members:

- Jeremy Agunda 150870
- Husna Hasan 138040
- Will Iha 151727
- Alvin Wade 151955

GitHub link: [BEE5102-IoT-Lab-Group-7](#)

Table of Contents

Using MicroPython	3
Introduction	3
Methodology	3
Hardware Setup	3
Software Tools	3
Step-by-Step Procedure	4
Our Code.....	4
main.py (ESP32 Code)	4
subscriber_db.py (PC Code)	5
Results.....	5
Challenges We Faced & How We Fixed Them.....	9
Conclusion	9
Appendix A: Source Code	9
main.py (ESP32).....	9
subscriber_db.py (PC)	11
mqtt_listener.py (PC)	12
Appendix B: Group Work Evidence	13

Using MicroPython

Introduction

This laboratory exercise aimed to develop a complete Internet of Things (IoT) device using the ESP32 microcontroller, a DHT22 temperature and humidity sensor, and the MQTT messaging protocol..

The primary objectives achieved in this lab include:

1. correctly wiring the DHT22 sensor to the ESP32 development board with a 10k ohm pull-up resist
2. flashing MicroPython firmware onto the ESP32 using esptool
3. writing MicroPython code to read sensor data, connect to WiFi, and publish structured JSON data to an MQTT broker
4. subscribing to the MQTT topic and persisting incoming data to a SQLite database
5. debugging the multi-layer IoT system end-to-end.

Methodology

Hardware Setup

The hardware assembly consisted of an ESP32-DevKitC development board, a DHT22 (AM2302) temperature and humidity sensor, a half-size breadboard, three male-to-male jumper wires, one 10kΩ pull-up resistor, and a USB data cable. The DHT22 sensor was wired to the ESP32 as follows:

DHT22 Pin	ESP32 Connection	Notes
VCC (Pin 1)	3.3V	Power supply
Data (Pin 2)	GPIO4	Digital data line
NC (Pin 3)	—	Not connected
GND (Pin 4)	GND	Ground reference

A 10kΩ pull-up resistor was placed between the DHT22 Data pin (GPIO4) and the 3.3V rail. This resistor is critical for reliable single-wire communication; without it, the DHT22 returns None readings or raises OSError exceptions. An annotated photograph of the breadboard circuit is provided in the Results section (Figure 1).

Software Tools

The following software stack was used throughout the lab:

Software	Version / Install Method	Purpose
Python	3.14 (64-bit)	PC-side scripting environment
Thonny IDE	Latest stable	MicroPython development and REPL
esptool	v5.3.1 (pip install)	Flash MicroPython firmware to ESP32
MicroPython	v1.29.0 (ESP32_GENERIC)	Firmware for ESP32
paho-mqtt	pip install paho-mqtt	Python MQTT client library
SQLite	Bundled with Python	Local relational database
HiveMQ Broker	Public (broker.hivemq.com)	Cloud MQTT message broker

Step-by-Step Procedure

Step 1: We put the ESP32 into download mode by holding the BOOT button while plugging in USB. Then we erased the old flash and wrote the new MicroPython firmware (ESP32_GENERIC v1.29.0) using `python -m esptool --port COM11 --baud 460800 write-flash -z 0x1000 firmware.bin`.

Step 2: We wired the DHT22 to the ESP32 on the breadboard, making sure the 10kΩ resistor was properly connected between the data line and 3.3V.

Step 3: In Thonny, we wrote `main.py` which connects to WiFi, reads the DHT22 every 5 seconds, and publishes the data as JSON to the MQTT topic `iot/lab/sensor`.

Step 4: We wrote `mqtt_listener.py` on the PC to check that messages were actually arriving at the broker. This was our way of verifying the ESP32 was working.

Step 5: We wrote `subscriber_db.py` which subscribes to the same topic, parses the JSON, and inserts rows into an SQLite database with timestamps. We then ran a SELECT query to confirm the data was saved.

Our Code

`main.py` (ESP32 Code)

This code runs on the ESP32. It does three main things: connects to WiFi, reads the DHT22 sensor, and sends the data to the MQTT broker. We used the `umqtt.simple`

library that comes with MicroPython to handle MQTT. The sensor is connected to GPIO4. We used `json.dumps()` to turn the temperature and humidity into a JSON string before sending it.

The code loops forever, reading the sensor every 5 seconds. If the sensor fails to read (which happens sometimes at startup), it prints an error and tries again. Once it gets good data, it publishes the JSON to the topic `iot/lab/sensor`.

`subscriber_db.py` (PC Code)

This script runs on the PC. It connects to the same MQTT broker and listens for messages on `iot/lab/sensor`. When a message arrives, it converts the JSON back into Python variables, gets the current time using `datetime.now()`, and inserts everything into an SQLite database. We used a `try/except` block so that if one message is corrupted, the program doesn't crash.

At the end, the script runs a `SELECT * FROM sensor_readings` query and prints all the rows so we can see that the data was actually saved.

Results

The system worked end-to-end. The ESP32 connected to WiFi (IP: 192.168.8.7), joined the MQTT broker, and started sending data every 5 seconds. The sensor readings were stable: temperature around 24.4–25.0°C and humidity around 49–50%.

```

MPY: soft reboot
Connecting to WiFi...
WiFi connected: 192.168.8.7
Connected to MQTT broker
Published: {"humidity": 49.5, "device": "esp32_lab_group", "temperature": 25.0}
Message #1 sent
Published: {"humidity": 49.600004, "device": "esp32_lab_group", "temperature": 25.0}
Message #2 sent
Published: {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 25.0}
Message #3 sent
Published: {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 25.0}
Message #4 sent
Published: {"humidity": 49.600004, "device": "esp32_lab_group", "temperature": 25.0}
Message #5 sent
Published: {"humidity": 49.8, "device": "esp32_lab_group", "temperature": 25.0}
Message #6 sent
Published: {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 25.0}
Message #7 sent
Published: {"humidity": 49.8, "device": "esp32_lab_group", "temperature": 25.0}
Message #8 sent
Published: {"humidity": 49.8, "device": "esp32_lab_group", "temperature": 25.0}
Message #9 sent
Published: {"humidity": 49.8, "device": "esp32_lab_group", "temperature": 25.0}
Message #10 sent
Published: {"humidity": 49.8, "device": "esp32_lab_group", "temperature": 25.0}
Message #11 sent
Published: {"humidity": 49.8, "device": "esp32_lab_group", "temperature": 24.9}
Message #12 sent
Published: {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.9}
Message #13 sent
Published: {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.9}
Message #14 sent
Published: {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.9}

```

Figure 1 Thonny REPL showing live sensor readings

```

-----
Message #15 sent
Published: {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.9}
Message #16 sent
Published: {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.9}
Message #17 sent
Published: {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.9}
Message #18 sent
Published: {"humidity": 50.0, "device": "esp32_lab_group", "temperature": 24.9}
Message #19 sent
Published: {"humidity": 50.0, "device": "esp32_lab_group", "temperature": 24.9}
Message #20 sent
Published: {"humidity": 50.0, "device": "esp32_lab_group", "temperature": 24.800002}
Message #21 sent
Published: {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.800002}
Message #22 sent
Published: {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 24.9}
Message #23 sent
Published: {"humidity": 49.5, "device": "esp32_lab_group", "temperature": 24.800002}
Message #24 sent
Published: {"humidity": 49.4, "device": "esp32_lab_group", "temperature": 24.800002}
Message #25 sent
Published: {"humidity": 49.5, "device": "esp32_lab_group", "temperature": 24.800002}
Message #26 sent
Published: {"humidity": 49.600004, "device": "esp32_lab_group", "temperature": 24.800002}
Message #27 sent
Published: {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 24.800002}
Message #28 sent
Published: {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 24.800002}
Message #29 sent
Published: {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 24.800002}
Message #30 sent
Published: {"humidity": 49.8, "device": "esp32_lab_group", "temperature": 24.800002}
-----

```

```

Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.
Active code page: 65001

C:\Users\User>python "C:\Users\User\Desktop\mqtt_listener.py"
C:\Users\User\Desktop\mqtt_listener.py:17: DeprecationWarning: Callback API version 1 is deprecated, update to latest vers
  client = mqtt.Client()
Connecting to broker.hivemq.com...
Connected to MQTT broker
Subscribed to: iot/lab/sensor

iot/lab/sensor {"temperature": 15.37, "humidity": 39.59, "pressure": 1008.15}
iot/lab/sensor {"humidity": 49.3, "device": "esp32_lab_group", "temperature": 24.5}
iot/lab/sensor {"humidity": 49.7, "device": "esp32_lab_group", "temperature": 24.6}
iot/lab/sensor {"humidity": 49.4, "device": "esp32_lab_group", "temperature": 24.5}
iot/lab/sensor {"humidity": 49.600004, "device": "esp32_lab_group", "temperature": 24.5}
iot/lab/sensor {"humidity": 50.0, "device": "esp32_lab_group", "temperature": 24.6}
iot/lab/sensor {"humidity": 49.9, "device": "esp32_lab_group", "temperature": 24.6}
iot/lab/sensor {"humidity": 49.5, "device": "esp32_lab_group", "temperature": 24.5}
iot/lab/sensor {"humidity": 49.5, "device": "esp32_lab_group", "temperature": 24.5}
iot/lab/sensor {"humidity": 49.4, "device": "esp32_lab_group", "temperature": 24.5}

```

Figure 2 MQTT listener showing 10+ JSON messages

```

Received: {'temperature': 15.37, 'humidity': 39.59, 'pressure': 1008.15}
[2026-09-07 17:01:06] Saved: 15.37C, 39.59%
Received: {'humidity': 49.600004, 'device': 'esp32_lab_group', 'temperature': 24.4}
[2026-09-07 17:01:07] Saved: 24.4C, 49.600004%
Received: {'humidity': 49.5, 'device': 'esp32_lab_group', 'temperature': 24.4}
[2026-09-07 17:01:12] Saved: 24.4C, 49.5%
Received: {'humidity': 49.4, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:01:17] Saved: 24.5C, 49.4%
Received: {'humidity': 49.600004, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:01:23] Saved: 24.5C, 49.600004%
Received: {'humidity': 49.8, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:01:28] Saved: 24.5C, 49.8%
Received: {'humidity': 49.600004, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:01:33] Saved: 24.5C, 49.600004%
Received: {'humidity': 49.600004, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:01:39] Saved: 24.5C, 49.600004%
Received: {'humidity': 49.7, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:01:44] Saved: 24.5C, 49.7%
Received: {'humidity': 49.600004, 'device': 'esp32_lab_group', 'temperature': 24.4}
[2026-09-07 17:01:49] Saved: 24.4C, 49.600004%
Received: {'humidity': 49.600004, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:01:54] Saved: 24.5C, 49.600004%
Received: {'humidity': 49.7, 'device': 'esp32_lab_group', 'temperature': 24.4}
[2026-09-07 17:02:00] Saved: 24.4C, 49.7%
Received: {'humidity': 49.7, 'device': 'esp32_lab_group', 'temperature': 24.5}
[2026-09-07 17:02:05] Saved: 24.5C, 49.7%

=== DATABASE CONTENTS ===
(13, '2026-09-07 17:02:05', 24.5, 49.7, 'esp32_lab_group')
(12, '2026-09-07 17:02:00', 24.4, 49.7, 'esp32_lab_group')
(11, '2026-09-07 17:01:54', 24.5, 49.600004, 'esp32_lab_group')
(10, '2026-09-07 17:01:49', 24.4, 49.600004, 'esp32_lab_group')
(9, '2026-09-07 17:01:44', 24.5, 49.7, 'esp32_lab_group')
(8, '2026-09-07 17:01:39', 24.5, 49.600004, 'esp32_lab_group')
(7, '2026-09-07 17:01:33', 24.5, 49.600004, 'esp32_lab_group')
(6, '2026-09-07 17:01:28', 24.5, 49.8, 'esp32_lab_group')
(5, '2026-09-07 17:01:23', 24.5, 49.600004, 'esp32_lab_group')
(4, '2026-09-07 17:01:17', 24.5, 49.4, 'esp32_lab_group')
(3, '2026-09-07 17:01:12', 24.4, 49.5, 'esp32_lab_group')
(2, '2026-09-07 17:01:07', 24.4, 49.600004, 'esp32_lab_group')

```

Figure 3 SQLite SELECT query showing saved data

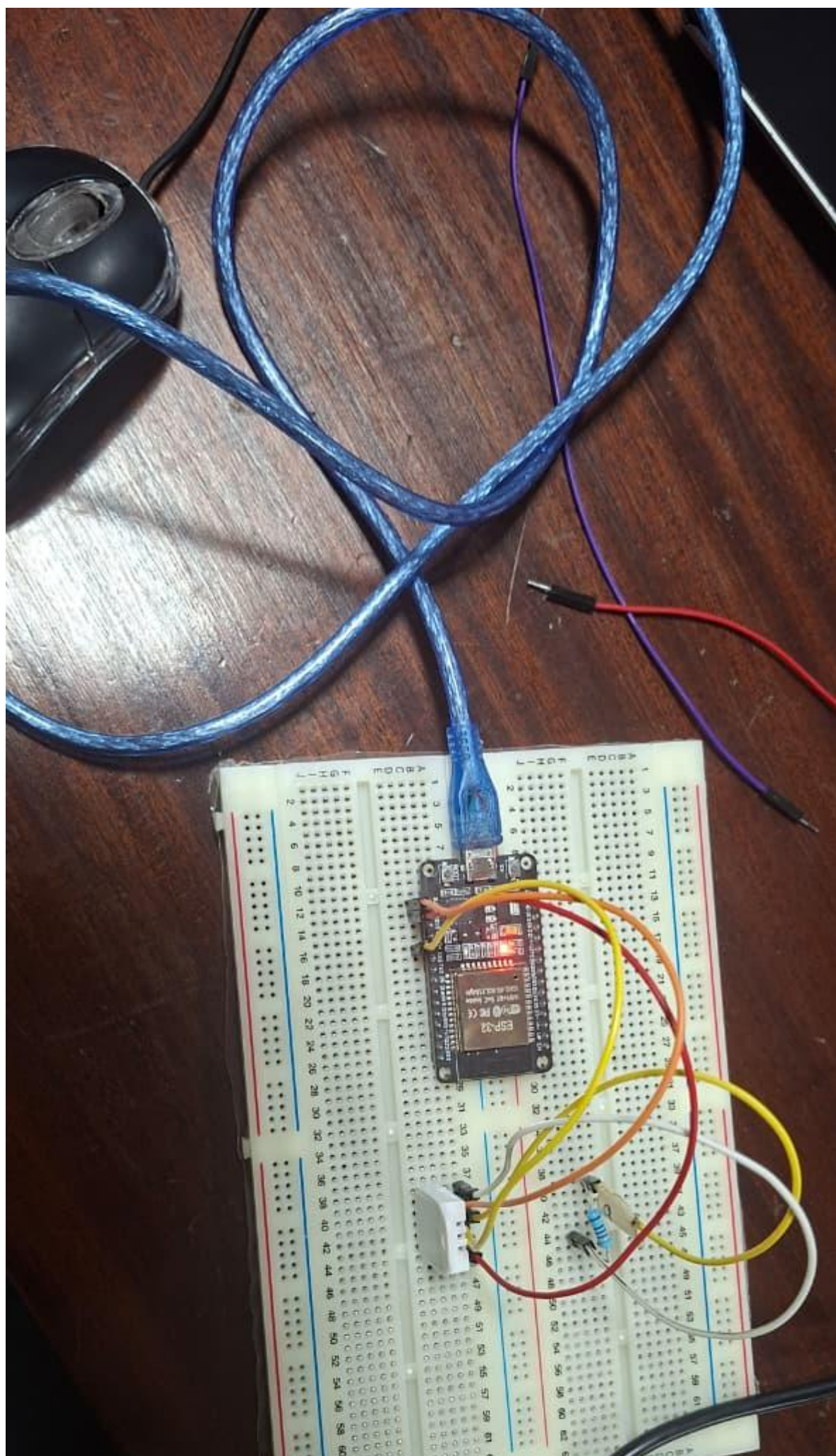


Figure 4 Breadboard wiring photo

Challenges We Faced & How We Fixed Them

This was our first time working with the ESP32 with MicroPython, so we ran into several problems. Below is a summary of what went wrong and how we solved it.

Conclusion

We successfully built a working IoT system from scratch. The ESP32 reads temperature and humidity from the DHT22, sends it over WiFi using MQTT, and a Python script on the PC saves that data to an SQLite database. We verified the whole pipeline works by checking the MQTT messages on the PC and by running a SELECT query that showed 12 rows of data with timestamps.

This lab taught us how the different parts of an IoT system fit together: the sensor (physical layer), the microcontroller (edge processing), WiFi and MQTT (network layer), and the database (application layer). We also learned how to debug problems by isolating which layer is failing whether it's wiring, code, or network connectivity.

For future improvements, we could maybe add a local LCD display to the ESP32 to show readings without needing a PC without many installations

Appendix A: Source Code

main.py (ESP32)

```
import machine
import dht
import network
import time
from umqtt.simple import MQTTClient
import json

WIFI_SSID = "Wifi"
WIFI_PASSWORD = "you12345"
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = b"iot/lab/sensor"

sensor = dht.DHT22(machine.Pin(4))

def connect_wifi():
    wlan = network.WLAN(network.S
```

```

TA_IF)
wlan.active(True)
if not
wlan.isconnected():
    print("Connecting to
WiFi...")
wlan.connect(WIFI_SSID,
WIFI_PASSWORD) while
not wlan.isconnected():
    time.sleep(1)
print("WiFi connected:",
wlan.ifconfig()[0]) return
wlan

def
read_
senso
r():
try:

sensor.measure()
temp =
sensor.temperature()
hum =
sensor.humidity()
return temp, hum
except Exception as
e:
print("Sensor read
error:", e)
return None, None

def publish_data(client, temp,
hum):
    payload =
json.dumps({
"temperature": temp,
    "humidity": hum,
    "device":
"esp32_lab_group"
})
client.publish(MQTT_TO
PIC, payload)
print("Published:",
payload)

def main():
    connect_wifi() client =
MQTTClient("esp32_client", MQTT_BROKER,
port=MQTT_PORT) client.connect()
print("Connected to MQTT broker") msg_count = 0
while True:
    temp, hum =
read_sensor() if temp
is not None and hum is not
None:
publish_data(client, temp, hum)
msg_count += 1
print(f"Message #{msg_count}
sent") else:
    print("Failed to read sensor,
retrying...") time.sleep(5)

if __name__ ==
"__main__":
main()

```

subscriber_db.py (PC)

```
import
paho.mqtt.clie
nt as mqtt
import sqlite3
import json
from datetime
import
datetime
import time

BROKER = "broker.hivemq.com"
TOPIC =
"iot/lab/senso
r" DB_NAME =
"sensor_data.d
b"

def init_db():
    conn = sqlite3.connect(DB_NAME)    cursor =
conn.cursor()    cursor.execute('''CREATE TABLE
IF NOT EXISTS sensor_readings (        id
INTEGER PRIMARY KEY AUTOINCREMENT,
timestamp TEXT NOT NULL,            temperature
REAL,            humidity REAL,            device
TEXT)''')    conn.commit()    conn.close()

def save_to_db(temp,
hum, device):
    conn =
sqlite3.connect(DB_NA
ME)    cursor =
conn.cursor()
    timestamp = datetime.now().strftime("%Y-
%m-%d %H:%M:%S")    cursor.execute('''INSERT
INTO sensor_readings        (timestamp,
temperature, humidity, device)        VALUES
(?, ?, ?, ?)''', (timestamp, temp, hum,
device))    conn.commit()    conn.close()
    print(f"[{timestamp}] Saved: {temp}C, {hum}%")

def on_connect(client, userdata, flags, rc,
properties=None):    if rc == 0:
        print("Connected
to MQTT broker")
    client.subscribe(TOPIC)
    print(f"Subscribed to:
{TOPIC}\n")    else:
        print(f"Connection
failed: {rc}")

def on_message(client,
userdata, msg):    try:
        payload =
json.loads(msg.payload.decode())
        temp = payload.get("temperature")
        hum = payload.get("humidity")
        device = payload.get("device",
"unknown")
        print(f"Received: {payload}")
        save_to_db(temp, hum, device)
    except Exception as e:
        print(f"Error: {e}")

def main():    init_db()    client =
mqtt.Client(mqtt.CallbackAPIVersion.VERSI
ON1)    client.on_connect = on_connect
    client.on_message = on_message
```

```

print(f"Connecting to {BROKER}")
client.connect(BROKER, 1883, 60)
client.loop_start()    print("Collecting
data for 60 seconds...")    try:

time.sleep(60)
except
KeyboardInterru
pt:
print("Stopping...")
finally:
    client.loop_stop()
client.disconnect()
print("\n=== DATABASE
CONTENTS ===")    conn
= sqlite3.connect(DB_NAME)
cursor = conn.cursor()
cursor.execute("SELECT *
FROM sensor_readings ORDER
BY timestamp DESC")
rows = cursor.fetchall()
for row in rows:
print(row)
conn.close()

if __name__ ==
"__main__":
main()

```

mqtt_listener.py (PC)

```

import paho.mqtt.client as mqtt

BROKER =
"broker.hivemq.com"
TOPIC =
"iot/lab/sensor"

def on_connect(client,
userdata, flags, rc):    if
rc == 0:
    print("Connected
to MQTT broker")
client.subscribe(TOPIC)
print(f"Subscribed to:
{TOPIC}\n")    else:
print(f"Connection
failed: {rc}")

def on_message(client,
userdata, msg):
print(f"{msg.topic}
{msg.payload.decode()}")

client =
mqtt.Client(mqtt.CallbackAPIVersion.VERSION1)
client.on_connect =
on_connect
client.on_message =
on_message

print(f"Connecting to {BROKER}...")
client.connect(BROKER, 1883, 60)
client.loop_forever()

```

Appendix B: Group Work Evidence

This lab was completed as a group project. All four members contributed to different parts of the work, from hardware assembly to writing code to preparing the report

